

Intro

Streamlit app can be found here: <https://andreas-fluckiger-mandatory-ind320.streamlit.app>

The Github repository can be found here: https://github.com/Fluckigerandreas/Mandatory_assignment_IND320.git

Some libraries and requirements for my notebook

```
In [ ]: import os
import pandas as pd
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, TimestampType, DoubleType, IntegerType
from datetime import datetime, timedelta
from dateutil.relativedelta import relativedelta

import os

# ✓ Java 11 path
os.environ["JAVA_HOME"] = "/Library/Java/JavaVirtualMachines/jdk-11.jdk/Contents/Home"

# ✓ Use your current Python environment for PySpark
os.environ["PYSPARK_PYTHON"] = "python"
os.environ["PYSPARK_DRIVER_PYTHON"] = "python"
```

```
In [ ]: from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Elhub Production to Cassandra") \
    .config("spark.jars.packages", "com.datastax.spark:spark-cassandra-connector_2.12:3.5.0") \
    .config("spark.cassandra.connection.host", "127.0.0.1") \
    .config("spark.cassandra.connection.port", "9042") \
    .config("spark.cassandra.auth.username", "cassandra") \
    .config("spark.cassandra.auth.password", "cassandra") \
    .config("spark.cassandra.output.consistency.level", "LOCAL_QUORUM") \
```

```

.config("spark.sql.extensions", "com.datastax.spark.connector.CassandraSparkExtensions") \
.getOrCreate()

print("✅ SparkSession successfully connected to Cassandra on localhost:9042")

```

Extracting data from API and adding to Cassandra

```

In [ ]: # %% [code]
import requests
import pandas as pd
from datetime import datetime
from dateutil.relativedelta import relativedelta

# --- API Settings ---
BASE_URL = "https://api.elhub.no/energy-data/v0/price-areas"
DATASET = "PRODUCTION_PER_GROUP_MBA_HOUR"
START_DATE = "2021-01-01"
END_DATE = "2024-12-31"

# --- Step 1: Get all available price areas dynamically ---
areas_response = requests.get(BASE_URL)
if areas_response.status_code == 200:
    areas = areas_response.json().get("priceAreas", ["N01", "N02", "N03", "N04", "N05"])
else:
    print("⚠️ Could not fetch price areas from API. Using defaults (N01-N05).")
    areas = ["N01", "N02", "N03", "N04", "N05"]

print(f"📌 Price areas to fetch: {areas}")

# --- Step 2: Function to fetch data for one area and time window ---
def fetch_data(area, start_date, end_date):
    api_url = f"{BASE_URL}/{area}"
    params = {
        "dataset": DATASET,
        "startDate": start_date,
        "endDate": end_date
    }

```

```

response = requests.get(api_url, params=params)

if response.status_code == 200:
    try:
        data = response.json()
        records = data["data"][0]["attributes"]["productionPerGroupMbaHour"]
        if records:
            df = pd.DataFrame(records)
            df["priceArea"] = area
            return df
        else:
            print(f"⚠ No data returned for {area}: {start_date} → {end_date}")
            return pd.DataFrame()
    except (KeyError, IndexError):
        print(f"⚠ Unexpected JSON structure for {area}: {start_date} → {end_date}")
        return pd.DataFrame()
else:
    print(f"⚠ Failed for {area}: {start_date} → {end_date} ({response.status_code})")
    return pd.DataFrame()

# --- Step 3: Loop over all months and all areas ---
all_data = []
start_dt = datetime.fromisoformat(START_DATE + "T00:00:00+02:00")
end_dt = datetime.fromisoformat(END_DATE + "T23:59:59+02:00")

for area in areas:
    current_start = start_dt
    while current_start < end_dt:
        current_end = current_start + relativedelta(months=1)
        if current_end > end_dt:
            current_end = end_dt

        start_str = current_start.isoformat()
        end_str = current_end.isoformat()

        print(f"Fetching {area}: {start_str} → {end_str}")
        df = fetch_data(area, start_str, end_str)
        if not df.empty:
            all_data.append(df)

```

```

        current_start = current_end

# --- Step 4: Combine and save all data ---
if all_data:
    final_df = pd.concat(all_data, ignore_index=True)
    final_df.to_csv("elhub_production_2021_all_areas.csv", index=False)
    print(f"✅ Data for all price areas saved to elhub_production_2021_all_areas.csv")
else:
    print("⚠️ No data collected.")

# %% [markdown]
# ## 2 Insert into Cassandra using Spark
from pyspark.sql.types import StructType, StructField, StringType, FloatType, DoubleType

schema = StructType([
    StructField("endTime", StringType(), True),
    StructField("lastUpdatedTime", StringType(), True),
    StructField("priceArea", StringType(), True),
    StructField("productionGroup", StringType(), True),
    StructField("quantityKwh", DoubleType(), True),
    StructField("starttime", StringType(), True),
])

# --- Convert Pandas DF to Spark DF ---
spark_df = spark.createDataFrame(final_df, schema=schema)

# --- Rename columns to match Cassandra primary key casing ---
spark_df = spark_df \
    .withColumnRenamed("endTime", "endtime") \
    .withColumnRenamed("lastUpdatedTime", "lastupdatedtime") \
    .withColumnRenamed("priceArea", "pricearea") \
    .withColumnRenamed("productionGroup", "productiongroup") \
    .withColumnRenamed("quantityKwh", "quantitykwh") \
    .withColumnRenamed("starttime", "starttime") \

# I ran this the first time, to make sure my code sent the data to cassandra. I also store a csv file locally on my

# --- Write to Cassandra ---
spark_df.write \
    .format("org.apache.spark.sql.cassandra") \

```

```
.mode("append") \  
.options(keyspace="elhub", table="production_total") \  
.save()
```

Extracting filtered data from Cassandra

```
In [ ]: from pyspark.sql import SparkSession  
from pyspark.sql.functions import col  
  
# Assuming SparkSession 'spark' is already created with Cassandra connector  
keyspace = "elhub"  
table = "production_total"  
  
# Load only the needed columns  
df = spark.read \  
    .format("org.apache.spark.sql.cassandra") \  
    .options(keyspace=keyspace, table=table) \  
    .load() \  
    .select("pricearea", "productiongroup", "starttime", "quantitykwh")  
  
# Convert to Pandas for plotting  
pdf = df.toPandas()
```

Adding dataframe from cassandra to MongoDB

```
In [ ]: import os, sys  
sys.path.append(os.getcwd())  
  
from mysecrets import USERNAME, PASSWORD  
from pymongo.mongo_client import MongoClient  
from pymongo.server_api import ServerApi  
import certifi  
  
# Use certifi for trusted CA certificate bundle  
ca = certifi.where()
```

```

# Construct the URI using the imported credentials
uri = f"mongodb+srv://{USERNAME}:{PASSWORD}@cluster0.chffuae.mongodb.net/?retryWrites=true&w=majority&appName=Clust

# Create client with TLS certificate
client = MongoClient(uri, server_api=ServerApi('1'), tls=True, tlsCAFile=ca)

# Ping the deployment
try:
    client.admin.command('ping')
    print("✅ Pinged your MongoDB deployment – connection successful!")
except Exception as e:
    print("❌ Connection failed:")
    print(e)

```

```

In [ ]: # Selecting a database and a collection
database = client['Elhub']
collection = database['Data']

try:
    # Convert DataFrame rows to a list of dictionaries
    data = pdf.to_dict("records")

    if data:
        collection.insert_many(data)
except:
    # Completely ignore any errors
    pass

```

Consumption part of data download

```

In [ ]: import os
import pandas as pd
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, IntegerType
from datetime import datetime, timedelta
from dateutil.relativedelta import relativedelta
import requests
from pyspark.sql.functions import col

```

```

# -----
#  Set environment variables
# -----

# Path to your Java 11 installation
os.environ["JAVA_HOME"] = "/Library/Java/JavaVirtualMachines/jdk-11.jdk/Contents/Home"

# Use the current Python for PySpark
os.environ["PYSPARK_PYTHON"] = "python"
os.environ["PYSPARK_DRIVER_PYTHON"] = "python"

# -----
#  Initialize SparkSession
# -----

from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("Elhub Production to Cassandra") \
    .config("spark.jars.packages", "com.datastax.spark:spark-cassandra-connector_2.12:3.5.0") \
    .config("spark.cassandra.connection.host", "127.0.0.1") \
    .config("spark.cassandra.connection.port", "9042") \
    .config("spark.cassandra.auth.username", "cassandra") \
    .config("spark.cassandra.auth.password", "cassandra") \
    .config("spark.cassandra.output.consistency.level", "LOCAL_QUORUM") \
    .config("spark.sql.extensions", "com.datastax.spark.connector.CassandraSparkExtensions") \
    .getOrCreate()

print( SparkSession successfully connected to Cassandra on localhost:9042")

```

```

In [ ]: # %% [code]
import requests
import pandas as pd
from datetime import datetime
from dateutil.relativedelta import relativedelta

# --- API Settings ---
BASE_URL = "https://api.elhub.no/energy-data/v0/price-areas"
DATASET = "CONSUMPTION_PER_GROUP_MBA_HOUR"
START_DATE = "2021-01-01"

```

```
END_DATE = "2024-12-31"

# --- Step 1: Get all available price areas dynamically ---
areas_response = requests.get(BASE_URL)
if areas_response.status_code == 200:
    areas = areas_response.json().get("priceAreas", ["N01", "N02", "N03", "N04", "N05"])
else:
    print("⚠ Could not fetch price areas from API. Using defaults (N01-N05).")
    areas = ["N01", "N02", "N03", "N04", "N05"]

print(f"📌 Price areas to fetch: {areas}")

# --- Step 2: Function to fetch data for one area and time window ---
def fetch_data(area, start_date, end_date):
    api_url = f"{BASE_URL}/{area}"
    params = {
        "dataset": DATASET,
        "startDate": start_date,
        "endDate": end_date
    }
    response = requests.get(api_url, params=params)

    if response.status_code == 200:
        try:
            data = response.json()
            records = data["data"][0]["attributes"]["consumptionPerGroupMbaHour"]
            if records:
                df = pd.DataFrame(records)
                df["priceArea"] = area
                return df
            else:
                print(f"⚠ No data returned for {area}: {start_date} → {end_date}")
                return pd.DataFrame()
        except (KeyError, IndexError):
            print(f"⚠ Unexpected JSON structure for {area}: {start_date} → {end_date}")
            return pd.DataFrame()
    else:
        print(f"⚠ Failed for {area}: {start_date} → {end_date} ({response.status_code})")
        return pd.DataFrame()
```



```

# --- Step 3: Loop over all months and all areas ---
all_data = []
start_dt = datetime.fromisoformat(START_DATE + "T00:00:00+02:00")
end_dt = datetime.fromisoformat(END_DATE + "T23:59:59+02:00")

for area in areas:
    current_start = start_dt
    while current_start < end_dt:
        current_end = current_start + relativedelta(months=1)
        if current_end > end_dt:
            current_end = end_dt

        start_str = current_start.isoformat()
        end_str = current_end.isoformat()

        print(f"Fetching {area}: {start_str} → {end_str}")
        df = fetch_data(area, start_str, end_str)
        if not df.empty:
            all_data.append(df)

        current_start = current_end

# --- Step 4: Combine and save all data ---
if all_data:
    final_df = pd.concat(all_data, ignore_index=True)
    final_df.to_csv("elhub_consumption_all_areas.csv", index=False)
    print(f"✅ Data for all price areas saved to elhub_production_2021_all_areas.csv")
else:
    print("⚠️ No data collected.")

# %% [markdown]
# ## 2 Insert into Cassandra using Spark
from pyspark.sql.types import StructType, StructField, StringType, DoubleType, IntegerType

# --- Define schema matching the Pandas DataFrame for consumption ---
schema = StructType([
    StructField("consumptionGroup", StringType(), True), # String
    StructField("endTime", StringType(), True), # String
    StructField("lastUpdatedTime", StringType(), True), # String
])

```

```

    StructField("meteringPointCount", IntegerType(), True), # Int
    StructField("priceArea", StringType(), True),          # String
    StructField("quantityKwh", DoubleType(), True),         # Float mapped to Double
    StructField("startTime", StringType(), True),           # String
])

# --- Convert Pandas DF to Spark DF ---
spark_df = spark.createDataFrame(final_df, schema=schema)

# --- Rename columns to match Cassandra table (all lowercase) ---
spark_df = spark_df \
    .withColumnRenamed("consumptionGroup", "consumptiongroup") \
    .withColumnRenamed("endTime", "endtime") \
    .withColumnRenamed("lastUpdatedTime", "lastupdatedtime") \
    .withColumnRenamed("meteringPointCount", "meteringpointcount") \
    .withColumnRenamed("priceArea", "pricearea") \
    .withColumnRenamed("quantityKwh", "quantitykwh") \
    .withColumnRenamed("startTime", "starttime")

# I ran this the first time, to make sure my code sent the data to cassandra. I also store a csv file locally on my

# --- Write to Cassandra ---
spark_df.write \
    .format("org.apache.spark.sql.cassandra") \
    .mode("append") \
    .options(keyspace="elhub", table="consumption_total") \
    .save()

```

```

In [ ]: from pyspark.sql import SparkSession
        from pyspark.sql.functions import col

# Assuming SparkSession 'spark' is already created with Cassandra connector
keyspace = "elhub"
table = "consumption_total"

# Load only the needed columns
df = spark.read \
    .format("org.apache.spark.sql.cassandra") \
    .options(keyspace=keyspace, table=table) \
    .load() \

```

```

        .select("pricearea", "consumptiongroup", "starttime", "quantitykwh")

# Convert to Pandas for plotting
pdf = df.toPandas()

```

```

In [ ]: # Use certifi for trusted CA certificate bundle
ca = certifi.where()

# Construct the URI using the imported credentials
uri = f"mongodb+srv://{USERNAME}:{PASSWORD}@cluster0.chffuae.mongodb.net/?retryWrites=true&w=majority&appName=Clust

# Create client with TLS certificate
client = MongoClient(uri, server_api=ServerApi('1'), tls=True, tlsCAFile=ca)

# Ping the deployment
try:
    client.admin.command('ping')
    print("✅ Pinged your MongoDB deployment – connection successful!")
except Exception as e:
    print("❌ Connection failed:")
    print(e)

```

```

In [ ]: # Selecting a database and a collection
database = client['Consumption_Elhub']
collection = database['Data']

try:
    # Convert DataFrame rows to a list of dictionaries
    data = pdf.to_dict("records")

    if data:
        collection.insert_many(data)
except:
    # Completely ignore any errors
    pass

```

Log

During the course of this project, several challenges arose in retrieving, processing, and visualizing data from the Elhub and Meteo APIs. One of the initial issues involved data extraction through Cassandra. When attempting to retrieve both production and consumption data simultaneously from Elhub, the queries did not return all expected records. This necessitated a segmented approach, handling production and consumption data separately to ensure completeness. While this added some complexity to the retrieval process, it ultimately enabled the creation of more reliable and consistent datasets for analysis.

In the Streamlit component, configuring the interactive map presented additional challenges. Achieving an accurate and visually meaningful color scheme required careful adjustments and extensive testing. Ensuring that the colors properly reflected the underlying data was crucial for the map to provide insightful visualizations.

For the SARIMAX model, the time series data from the APIs was resampled from an hourly to a daily frequency. This resampling allowed the model to execute without triggering timeouts in Streamlit, reducing computational load while preserving key trends and patterns necessary for reliable forecasting.

Another operational challenge occurred when Streamlit temporarily took the web application offline due to heavy data loading. To address this, a global caching mechanism was implemented for all Elhub data. With caching in place, repeated requests no longer required full reloads, significantly improving loading times and responsiveness. Meteo API data, however, is not cached, so some delays remain. Despite this, caching Elhub data has noticeably enhanced the stability and usability of the application.

All visualizations within the Streamlit app were migrated from Matplotlib to Plotly, bringing substantial benefits such as interactive zooming and panning, improved rendering performance, and enhanced user experience. This interactivity allows users to explore trends and patterns in the data more effectively. Regarding the bonus task, I computed the monthly snowdrift and implemented some caching functions, though I am unsure if the caching fully meets the expectations for this task.

In terms of correlations, I observed a stronger relationship between higher temperatures and solar energy production, which aligns with expectations. No extreme weather events were recorded in Ås during 2023, so no significant anomalies were detected in that regard.

Finally, both setups used to download all relevant Elhub and Meteo data are documented in the accompanying PDF. Overall, the combination of segmented data retrieval, resampling for modeling, selective caching, and interactive visualizations has improved both the performance and usability of the application, enabling more reliable insights from the API datasets.

AI Assistance and Collaboration

Throughout the assignment, both ChatGPT and Perplexity were used extensively to support coding, debugging, and understanding the tasks. Key contributions from these AI tools included:

- Error troubleshooting: Identifying causes of module import errors, resolving Streamlit deprecation warnings, and clarifying unexpected behavior in code execution.
- Code refactoring: Assisting in combining CSV reading, API fetching, and plotting logic into a single, streamlined, and interactive application.
- UX and performance improvements: Recommending appropriate caching strategies and other design adjustments to handle large datasets, such as full-year hourly data, more efficiently.
- Task comprehension: Helping to understand complex requirements, enabling more effective planning and implementation of the project workflow.

In addition to AI support, collaboration with another student was instrumental in discussing approaches, testing solutions, and validating results. While iterative prompting was sometimes required to refine AI-generated suggestions, using these tools substantially accelerated development, reduced debugging time, and clarified technical design decisions. This combination of AI assistance and peer collaboration allowed me to focus on creating a responsive, user-friendly, and fully interactive Streamlit application.